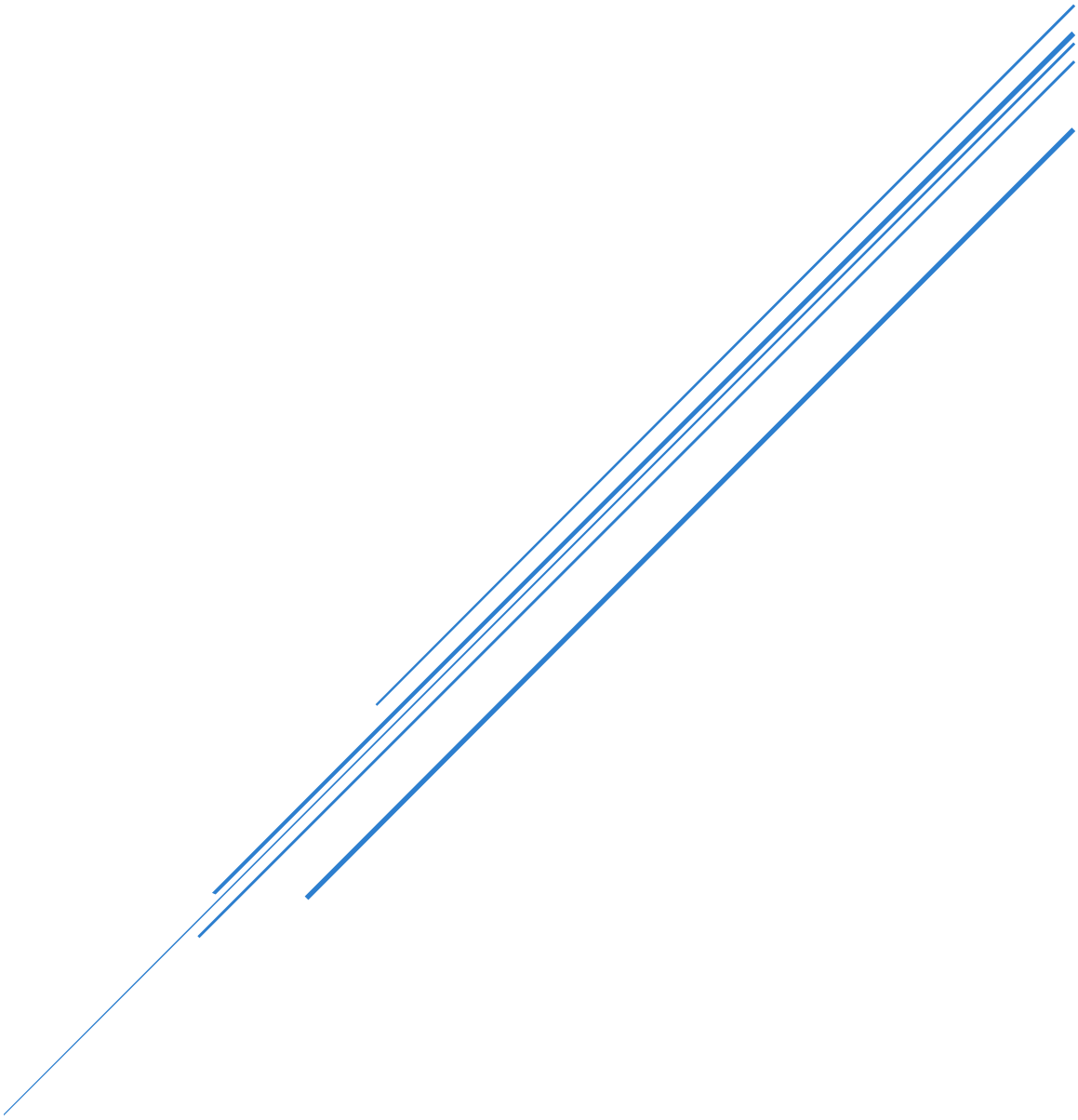
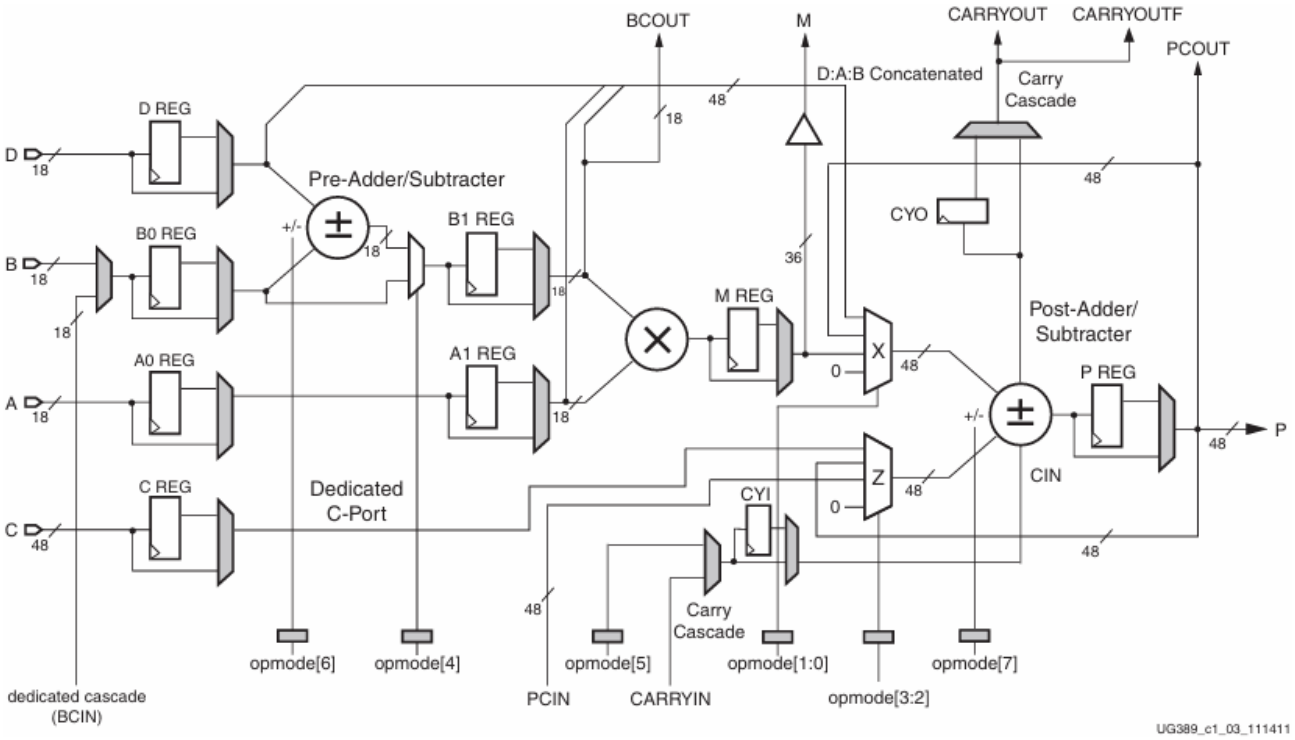
SPARTAN6 - DSP48A1

By : Mazen Mohamed Hemdan





Source of the requirements: LINK

1) RTL CODE :

- Port and Parameters Initialization

```
module DSP48A1 #(
    parameter A0REG = 0,
    parameter A1REG = 1,
    parameter B0REG = 0,
    parameter B1REG = 1,
    parameter CREG = 1,
    parameter DREG = 1,
    parameter MREG = 1,
    parameter PREG = 1,
    parameter CARRYINREG = 1,
    parameter CARRYOUTREG = 1,
    parameter OPMODEREG = 1,
    parameter CARRYINSEL = "OPMODE5",
    parameter B_INPUT = "DIRECT",
    parameter RSTTYPE = "SYNC"
)(
    input clk,CARRYIN,
    input CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP,
    input RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP,
    input [17:0] A,B,D,BCIN,
    input [47:0] C,PCIN,
    input [7:0] opmode,
    output CARRYOUT,CARRYOUTF,
    output [17:0] BCOUT,
    output [35:0] M,
    output [47:0] P,PCOUT
);
```

- Wires

```
wire [17:0] B_mux,mux_D_out,mux_B0_out,mux_A0_out,pre_AddSub_out,B1,mux_A1_out;
wire [47:0] mux_C_out;
wire [7:0] mux_opmode_out;
wire carrycascade,mux_CYI_out,post_CARRYOUT;
wire [35:0] M_out;
wire [47:0] post_AddSub_out;
reg [47:0] X,Z;
```

- All 5 Stages

```
// Stage 1
assign B_mux = (B_INPUT == "CASCADE")? BCIN : B;
Reg_Mux #(RSTTYPE,DREG) D_Reg (clk,RSTD,CED,D,mux_D_out);
Reg_Mux #(RSTTYPE,B0REG) B0_Reg (clk,RSTB,CEB,B_mux,mux_B0_out);
Reg_Mux #(RSTTYPE,A0REG) A0_Reg (clk,RSTA,CEA,A,mux_A0_out);
Reg_Mux #(RSTTYPE,CREG,48) C_Reg (clk,RSTC,CEC,C,mux_C_out);
Reg_Mux #(RSTTYPE,OPMODEREG,8) opmode_reg (clk,RSTOPMODE,CEOPMODE,opmode,mux_opmode_out);

// Stage 2
AddSub Pre_AddSub (mux_D_out,mux_B0_out,0,mux_opmode_out[6],pre_AddSub_out);
assign B1 = (mux_opmode_out[4])? pre_AddSub_out : mux_B0_out;
Reg_Mux #(RSTTYPE,B1REG) B1_Reg (clk,RSTB,CEB,B1,BCOUT);
Reg_Mux #(RSTTYPE,A1REG) A1_Reg (clk,RSTA,CEA,mux_A0_out,mux_A1_out);

// Stage 3
MULT mult(BCOUT,mux_A1_out,M_out);
Reg_Mux #(RSTTYPE,MREG,36) M_Reg (clk,RSTM,CEM,M_out,M);
assign carrycascade = (CARRYINSEL == "OPMODE5")? mux_opmode_out[5] : CARRYIN ;
Reg_Mux #(RSTTYPE,CARRYINREG,1) CYI (clk,RSTCARRYIN,CECARRYIN,carrycascade,mux_CYI_out);

// Stage 4
always@(*)begin
    case(mux_opmode_out[1:0])
        2'b11 : X = {mux_D_out[11:0],mux_A1_out,BCOUT};
        2'b10 : X = PCOUT;
        2'b01 : X = {12'b0, M};
        2'b00 : X = 0;
        default : X = 0;
    endcase
end
always@(*)begin
    case(mux_opmode_out[3:2])
        2'b11 : Z = mux_C_out;
        2'b10 : Z = PCOUT;
        2'b01 : Z = PCIN;
        2'b00 : Z = 0;
        default : Z = 0;
    endcase
end

// Stage 5
AddSub #(48) Post_AddSub (Z,X,mux_CYI_out,mux_opmode_out[7],post_AddSub_out,post_CARRYOUT);
Reg_Mux #(RSTTYPE,CARRYOUTREG,1) CYO (clk,RSTCARRYIN,CECARRYIN,post_CARRYOUT,CARRYOUT);
assign CARRYOUTF = CARRYOUT;
Reg_Mux #(RSTTYPE,PREG,48) P_Reg (clk,RSTP,CEP,post_AddSub_out,P);
assign PCOUT = P;

endmodule
```

- Register followed by Mux

```
module Reg_Mux(clk,rst,CE,d,out);
parameter RSTTYPE = "SYNC";
parameter REG = 1;
parameter WIDTH = 18;
input clk,rst,CE;
input [WIDTH-1:0]d;
reg [WIDTH-1:0]out_reg;
output [WIDTH-1:0]out;
generate
        if(RSTTYPE == "SYNC")begin
            always@(posedge clk)begin
                if(rst)out_reg<=0;
                else if(CE)out_reg<=d;
            end
        end
        else if(RSTTYPE == "ASYNC")begin
            always@(posedge clk or posedge rst)begin
                if(rst)out_reg<=0;
                else if(CE)out_reg<=d;
            end
        end
endgenerate
assign out = (REG)? out_reg : d;
endmodule
```

- Add/Sub

```
module AddSub(in0,in1,cin,opmode,sum,carryout);
parameter WIDTH = 18;
input [WIDTH-1:0]in0,in1;
input opmode,cin;
output [WIDTH-1:0]sum;
output carryout;
assign {carryout,sum} = (opmode)? (in0-(in1+cin)) : (in0+in1+cin);
endmodule
```

- Multiplier

```
module MULT(in0,in1,out);
parameter WIDTH = 18;
input [WIDTH-1:0]in0,in1;
output [(WIDTH*2)-1:0]out;
assign out = in0 * in1;
endmodule
```

2) TESTBENCH CODE :

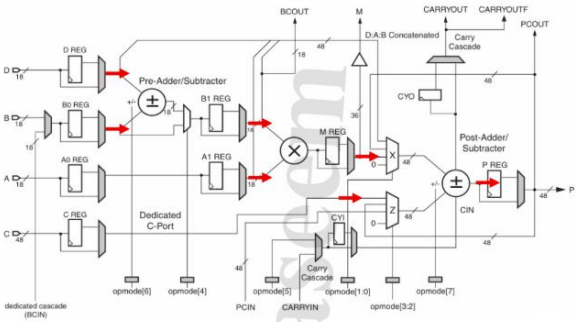
- module instantiation and all required reg and wires

```
module DSP48A1_tb;
    reg clk;
    reg CARRYIN;
    reg CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP;
    reg RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP;
    reg [17:0] A, B, D, BCIN;
    reg [47:0] C, PCIN;
    reg [7:0]  opmode;
    wire CARRYOUT, CARRYOUTF;
    wire [17:0] BCOUT;
    wire [35:0] M;
    wire [47:0] P, PCOUT;
    reg CARRYOUT_old;
    reg [47:0] P_old;
    DSP48A1 DUT (
        .clk(clk),
        .CARRYIN(CARRYIN),
        .CEA(CEA), .CEB(CEB), .CEC(CEC), .CECARRYIN(CECARRYIN),
        .CED(CED), .CEM(CEM), .CEOPMODE(CEOPMODE), .CEP(CEP),

        .RSTA(RSTA), .RSTB(RSTB), .RSTC(RSTC), .RSTCARRYIN(RSTCARRYIN),
        .RSTD(RSTD), .RSTM(RSTM), .RSTOPMODE(RSTOPMODE), .RSTP(RSTP),

        .A(A), .B(B), .D(D), .BCIN(BCIN),
        .C(C), .PCIN(PCIN),
        .opmode(opmode),

        .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF),
        .BCOUT(BCOUT),
        .M(M),
        .P(P), .PCOUT(PCOUT)
    );
```

- CLK configuration and verify reset operation

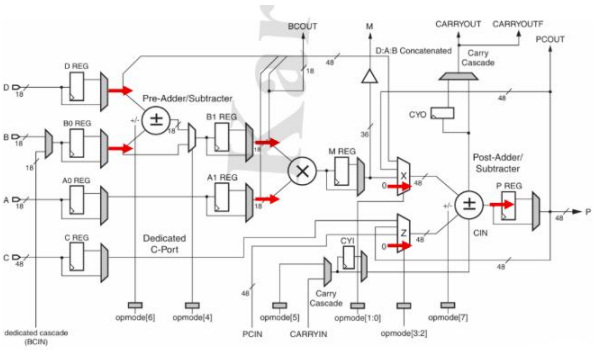
```
initial begin
        clk = 0;
        forever #1 clk = ~clk;
    end
    // 2. Stimulus Generation
    initial begin
        // 2.1. Verify Reset Operation
        // Assert all resets
        RSTA = 1; RSTB = 1; RSTC = 1; RSTCARRYIN = 1;
        RSTD = 1; RSTM = 1; RSTOPMODE = 1; RSTP = 1;

        // Drive remaining inputs with random values
        A = $random;B = $random;D = $random;BCIN = $random;
        C = $random;PCIN = $random;opmode = $random;CARRYIN = $random;
        CEA = $random;CEB = $random;CEC = $random;CECARRYIN = $random;
        CED = $random;CEM = $random;CEOPMODE = $random;CEP = $random;

        // Wait for negative edge of the clock
        @(negedge clk);

        // Self Checking
        if (CARRYOUT != 0 || CARRYOUTF != 0 || BCOUT != 0 ||
            M != 0 || P != 0 || PCOUT != 0) begin
            $display("Outputs are not zero during reset");
            $stop;
            end

        // Deassert all resets
        RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0;
        RSTD = 0; RSTM = 0; RSTOPMODE = 0; RSTP = 0;
        CEA = 1;CEB = 1;CEC = 1;CECARRYIN = 1;
        CED = 1;CEM = 1;CEOPMODE = 1;CEP = 1;
```

- Verify DSP Path 1



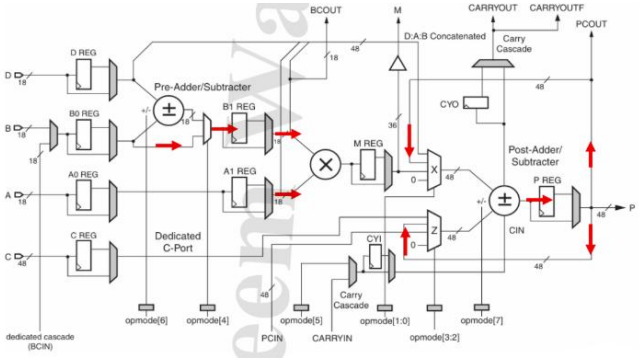

```verilog
// 2.2. Verify DSP Path 1
        opmode = 8'b11011101;
        A = 20; B = 10; C = 350; D = 25;
        BCIN = $random; PCIN = $random; CARRYIN = $random;
        repeat(4)@(negedge clk);
        if (BCOUT !== 18'hf || M !== 36'h12c || P !== 48'h32 ||
            PCOUT !== 48'h32 || CARRYOUT !== 1'b0 || CARRYOUTF !== 1'b0) begin
            $display("Outputs are wrong for path 1");
            $stop;
            end
```

- Verify DSP Path 2



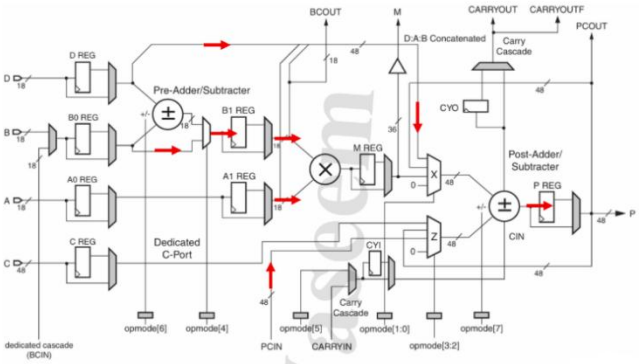

```verilog
// 2.3. Verify DSP Path 2
        opmode = 8'b00010000;
        A = 20; B = 10; C = 350; D = 25;
        BCIN = $random; PCIN = $random; CARRYIN = $random;
        repeat(3)@(negedge clk);
        if (BCOUT !== 18'h23 || M !== 36'h2bc || P !== 48'h0 ||
            PCOUT !== 48'h0 || CARRYOUT !== 1'b0 || CARRYOUTF !== 1'b0) begin
            $display("Outputs are wrong for path 2");
            $stop;
            end
```

- Verify DSP Path 3




```verilog
// 2.4. Verify DSP Path 3
        opmode = 8'b00001010;
        A = 20; B = 10; C = 350; D = 25;
        BCIN = $random; PCIN = $random; CARRYIN = $random;
        P_old = P; CARRYOUT_old = CARRYOUT;
        repeat(3)@(negedge clk);
        if (BCOUT !== 18'ha || M !== 36'hc8 || P !== P_old ||
            PCOUT !== P_old || CARRYOUT !== CARRYOUT_old || CARRYOUTF !== CARRYOUT_old) begin
            $display("Outputs are wrong for path 3");
            $stop;
            end
```

- Verify DSP Path 4




```verilog
// 2.5. Verify DSP Path 4
        opmode = 8'b10100111;
        A = 5; B = 6; C = 350; D = 25; PCIN = 3000;
        BCIN = $random; CARRYIN = $random;
        P_old = P; CARRYOUT_old = CARRYOUT;
        repeat(3)@(negedge clk);
        if (BCOUT !== 18'h6 || M !== 36'h1e || P !== 48'hfe6fffec0bb1 ||
            PCOUT !== 48'hfe6fffec0bb1 || CARRYOUT !== 1 || CARRYOUTF !== 1) begin
            $display("Outputs are wrong for path 4");
            $stop;
            end

        $stop;
    end

endmodule
```

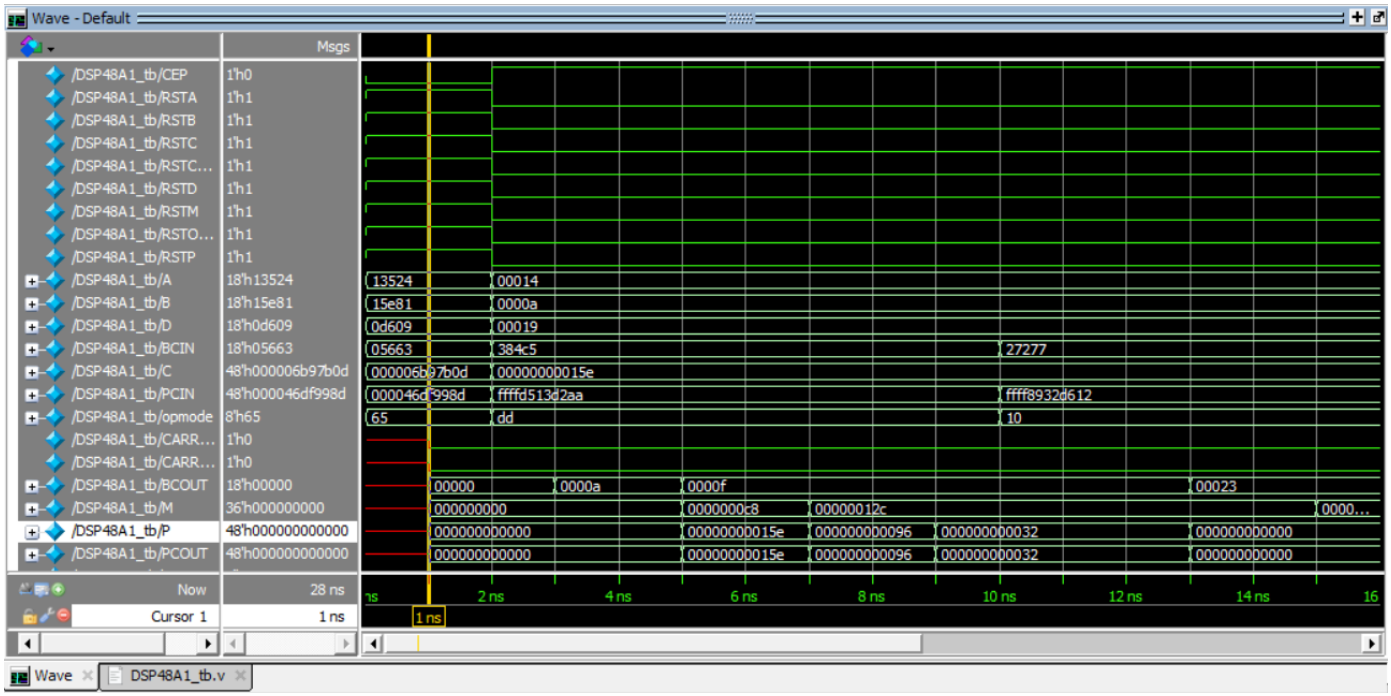
3) DO FILE :

```
vlib work
vlog DSP48A1.v DSP48A1_tb.v Reg_Mux.v AddSub.v MULT.v
vsim -voptargs=+acc work.DSP48A1_tb
add wave *
run -all
#quit -sim
```

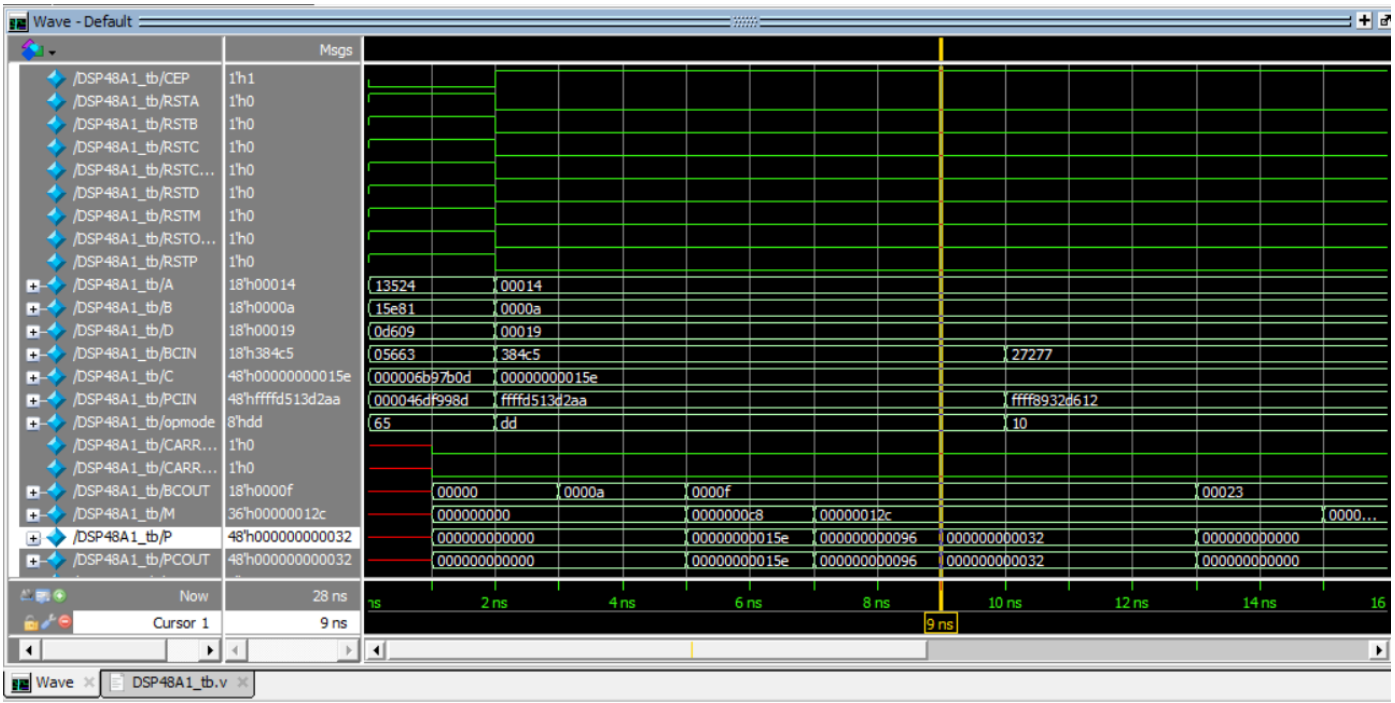
```
VSIM 13> do RUN_DSP48A1.do
# ** Warning: (vlib-34) Library already exists at "work".
# Errors: 0, Warnings: 1
# QuestaSim-64 vlog 2021.1 Compiler 2021.01 Jan 19 2021
# Start time: 04:32:25 on Jul 25,2025
# vlog -reportprogress 300 DSP48A1.v DSP48A1_tb.v Reg_Mux.v AddSub.v MULT.v
# -- Compiling module DSP48A1
# -- Compiling module DSP48A1_tb
# -- Compiling module Reg_Mux
# -- Compiling module AddSub
# -- Compiling module MULT
#
# Top level modules:
#       DSP48A1_tb
# End time: 04:32:25 on Jul 25,2025, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

4) QUESTASIM SNIPPETS :

- 2.1. Verify Reset Operation

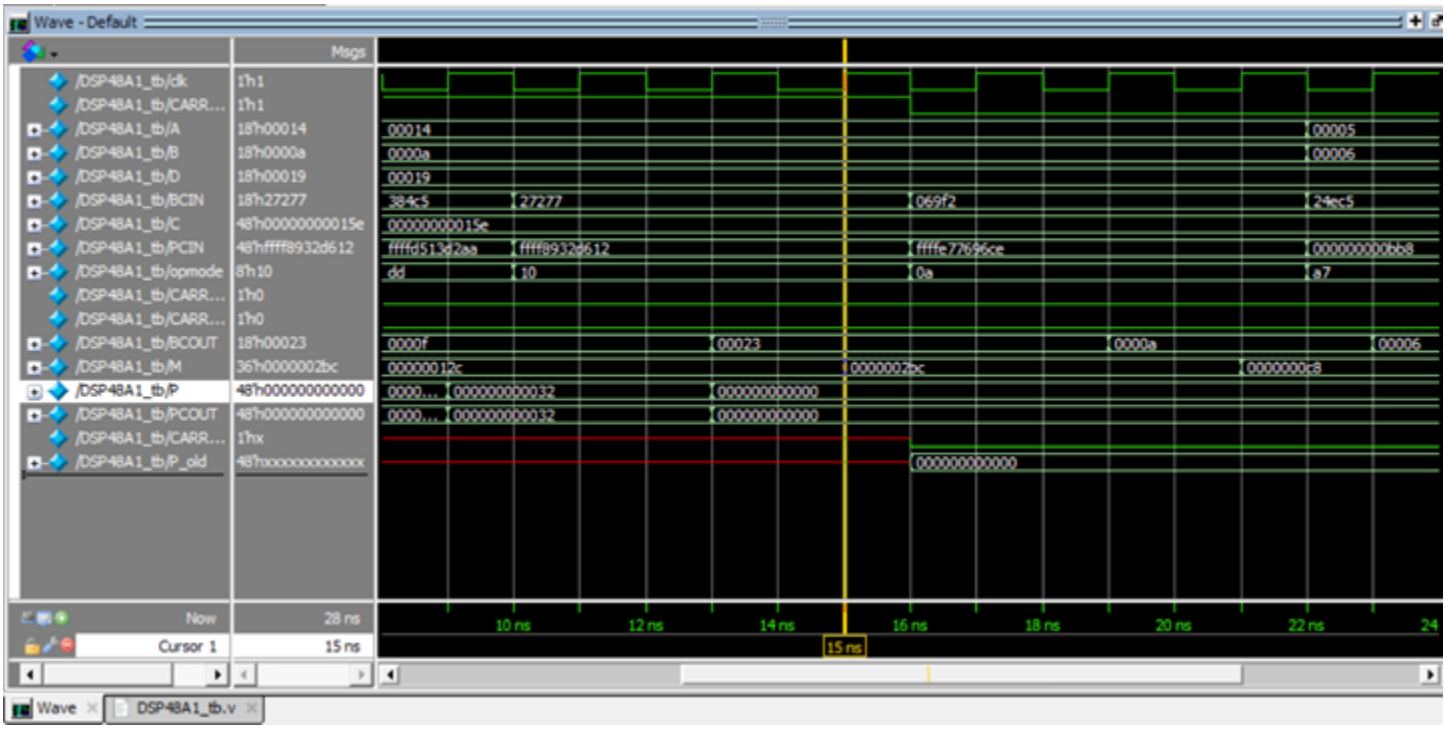


- 2.2. Verify DSP Path 1

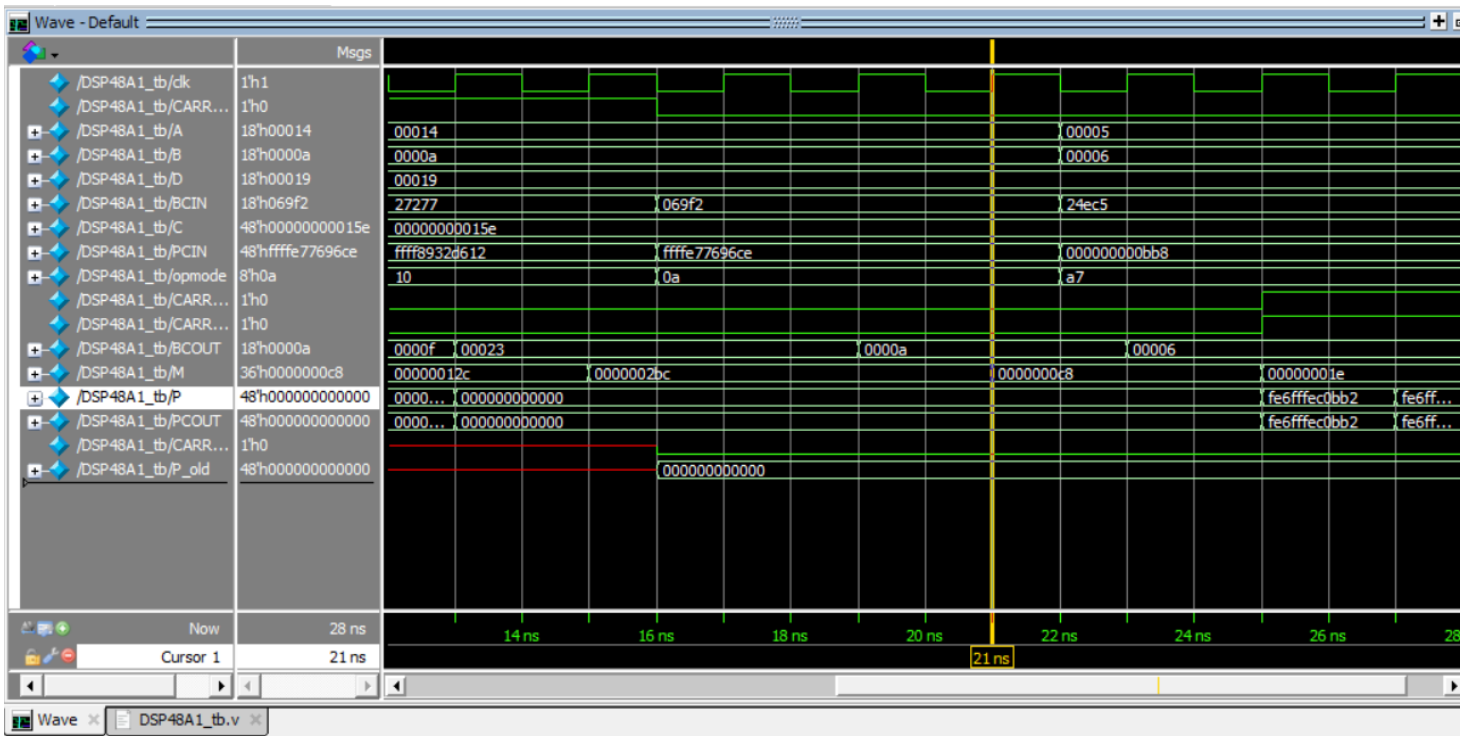


- 2.3. Verify DSP Path 2

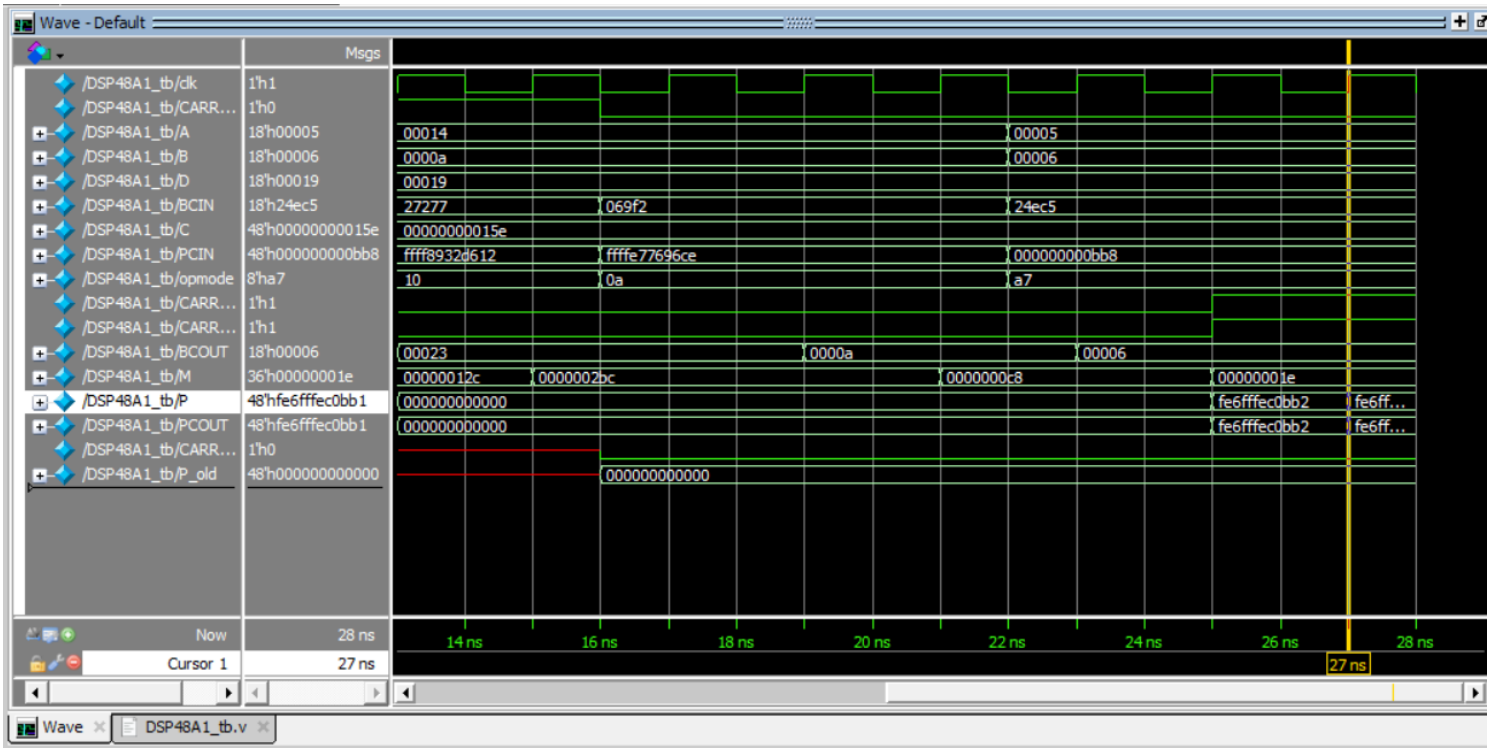
- Removed the rst and CE ports for more clear view



- 2.4. Verify DSP Path 3



- 2.5. Verify DSP Path 4



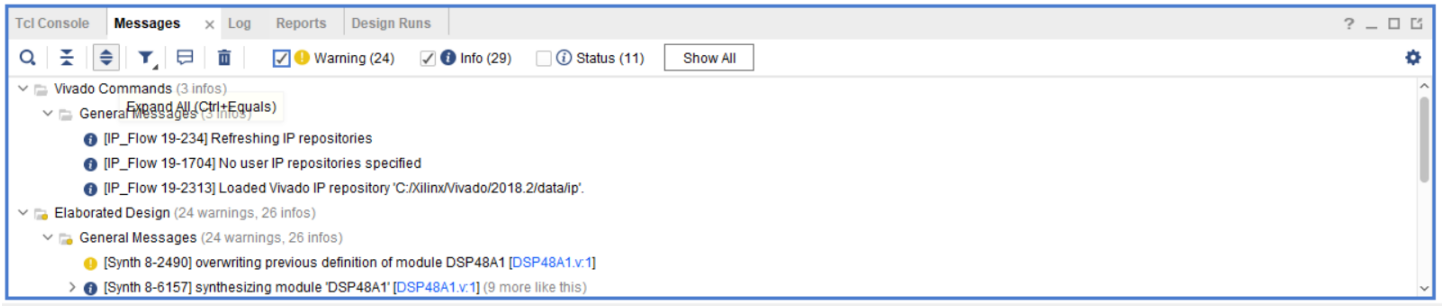
5) CONSTRAINT FILE :

```
## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

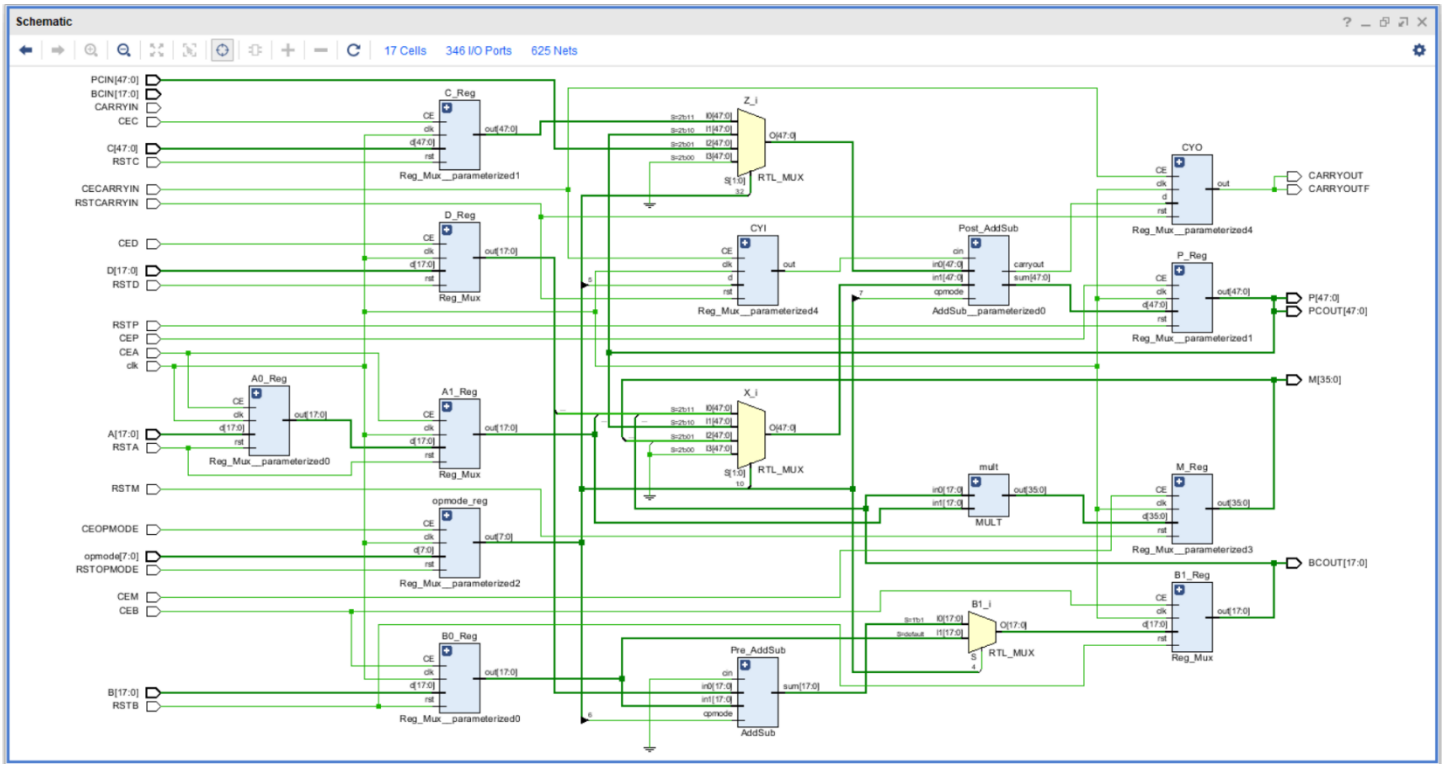
- Defines the clock frequency to 100 MHz to pin W5

6) ELABORATION :

- Messages



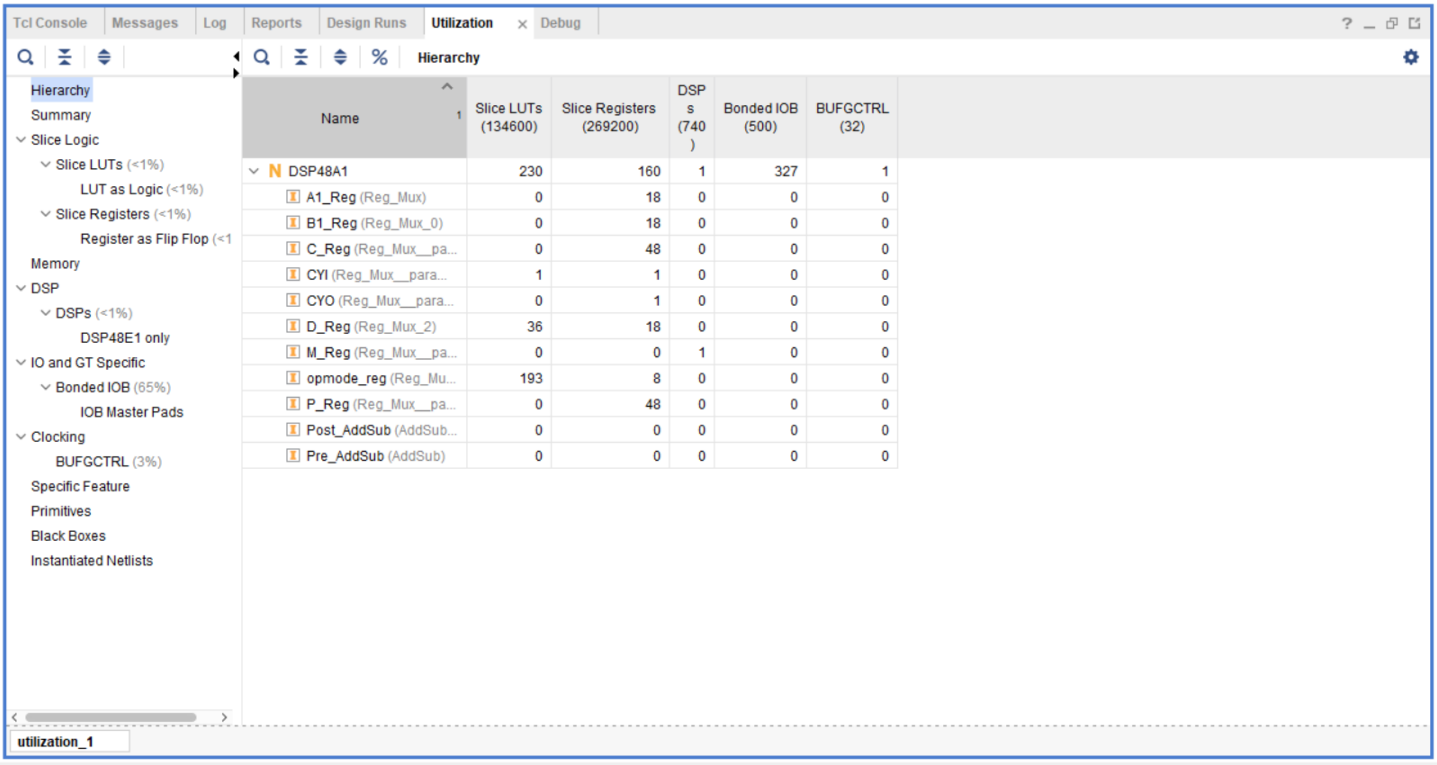
- Schematic Snippets



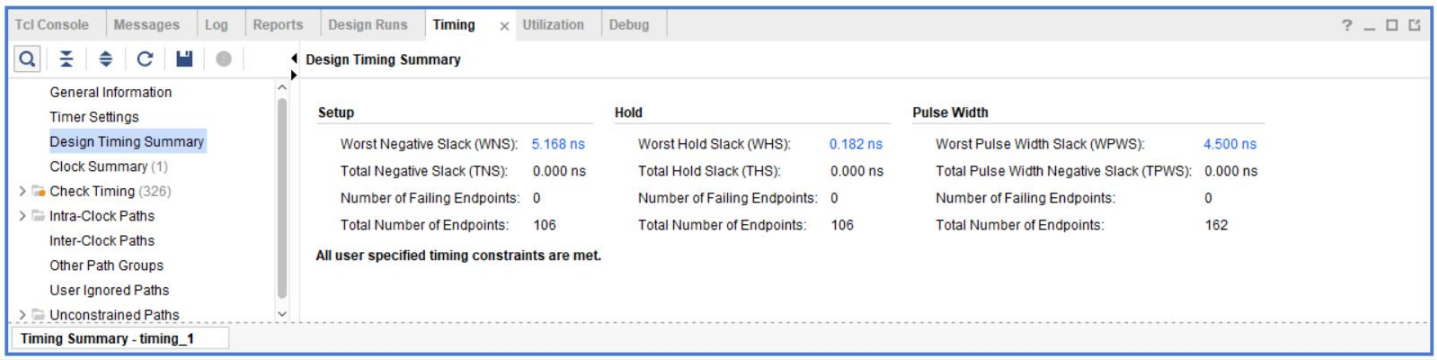
7) SYNTHESIS :

- Messages

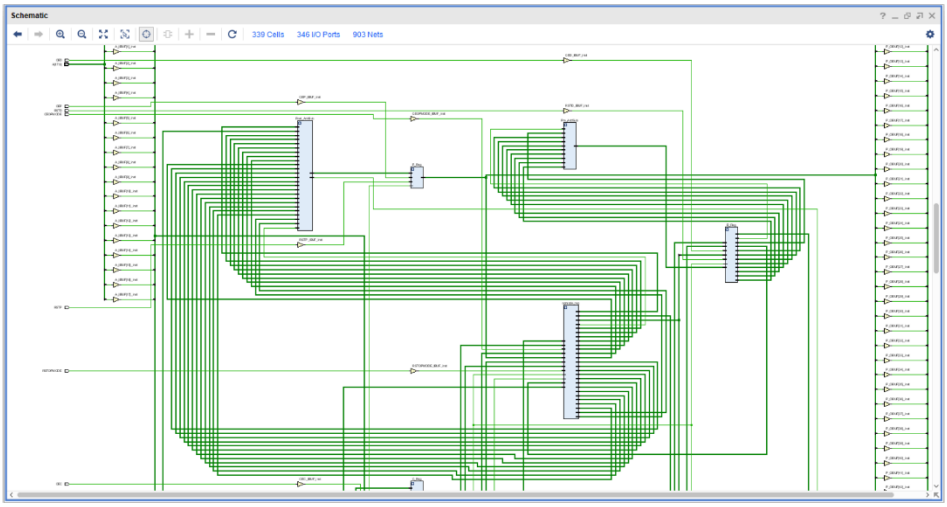
- Utilization Report

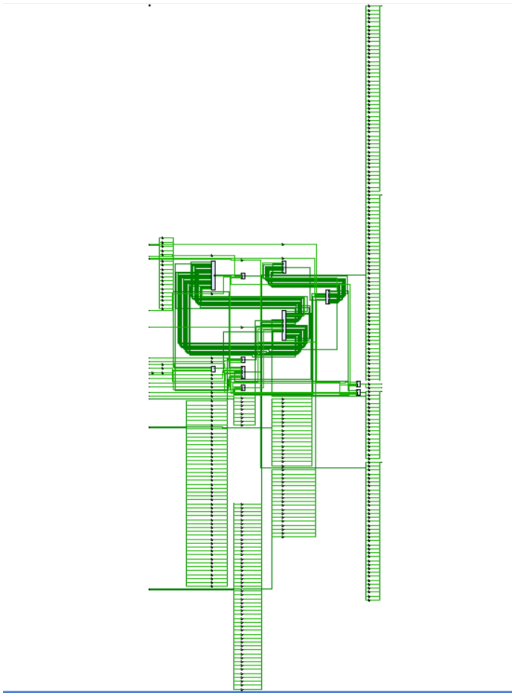


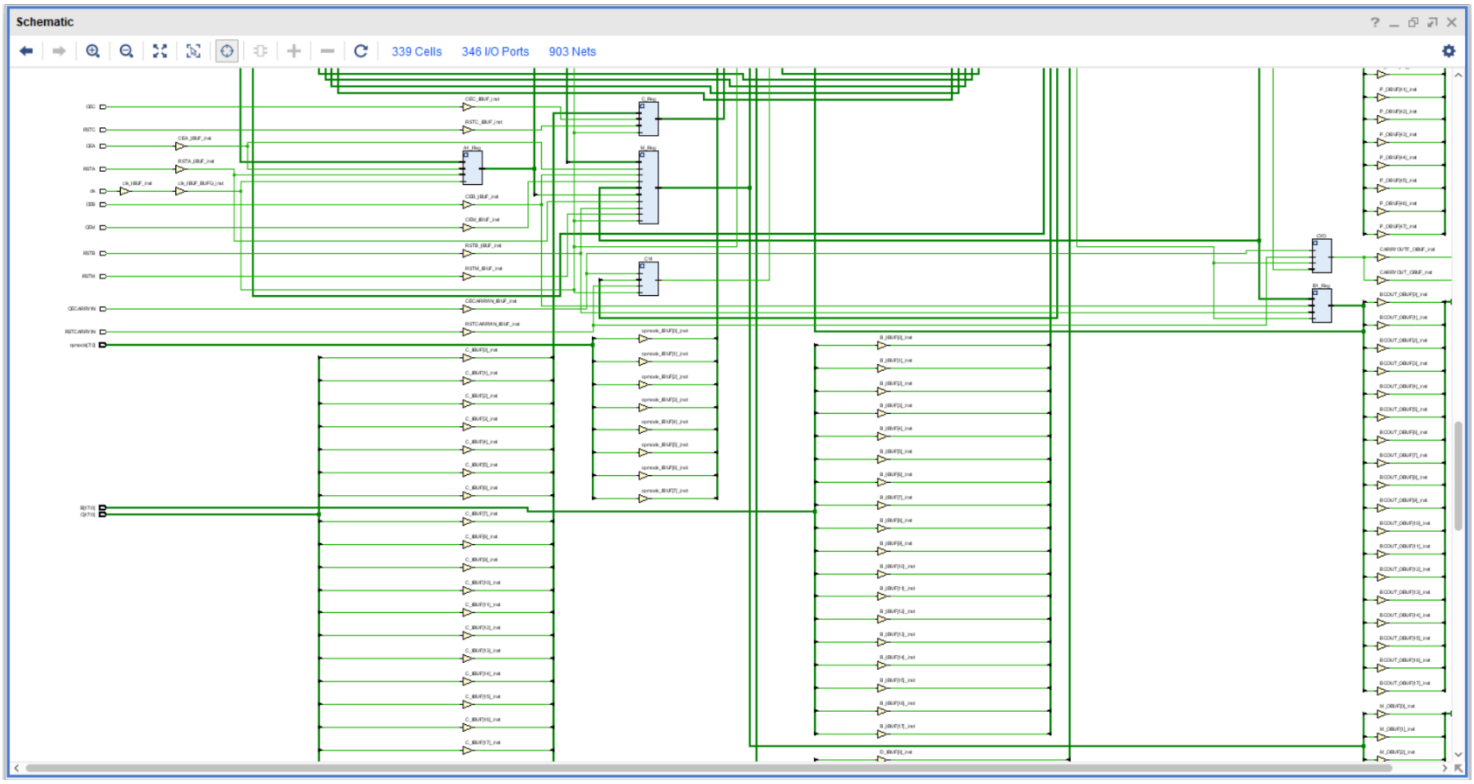
- Timing Report



- Schematic Snippets

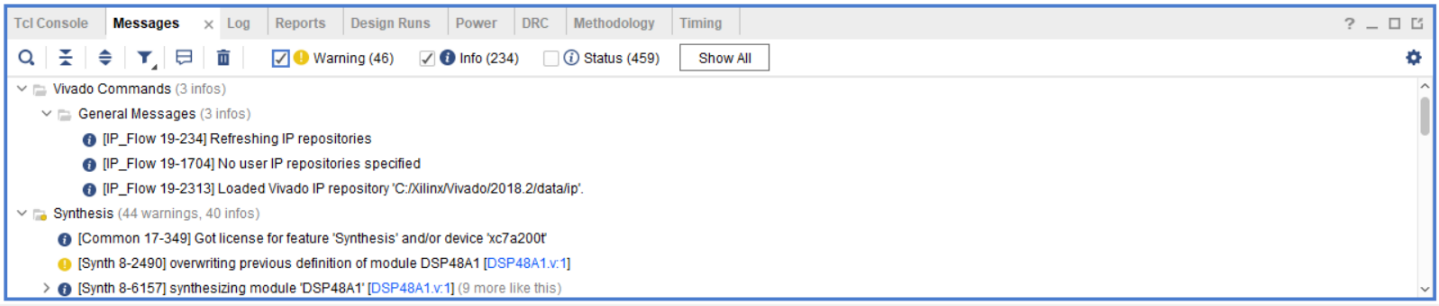




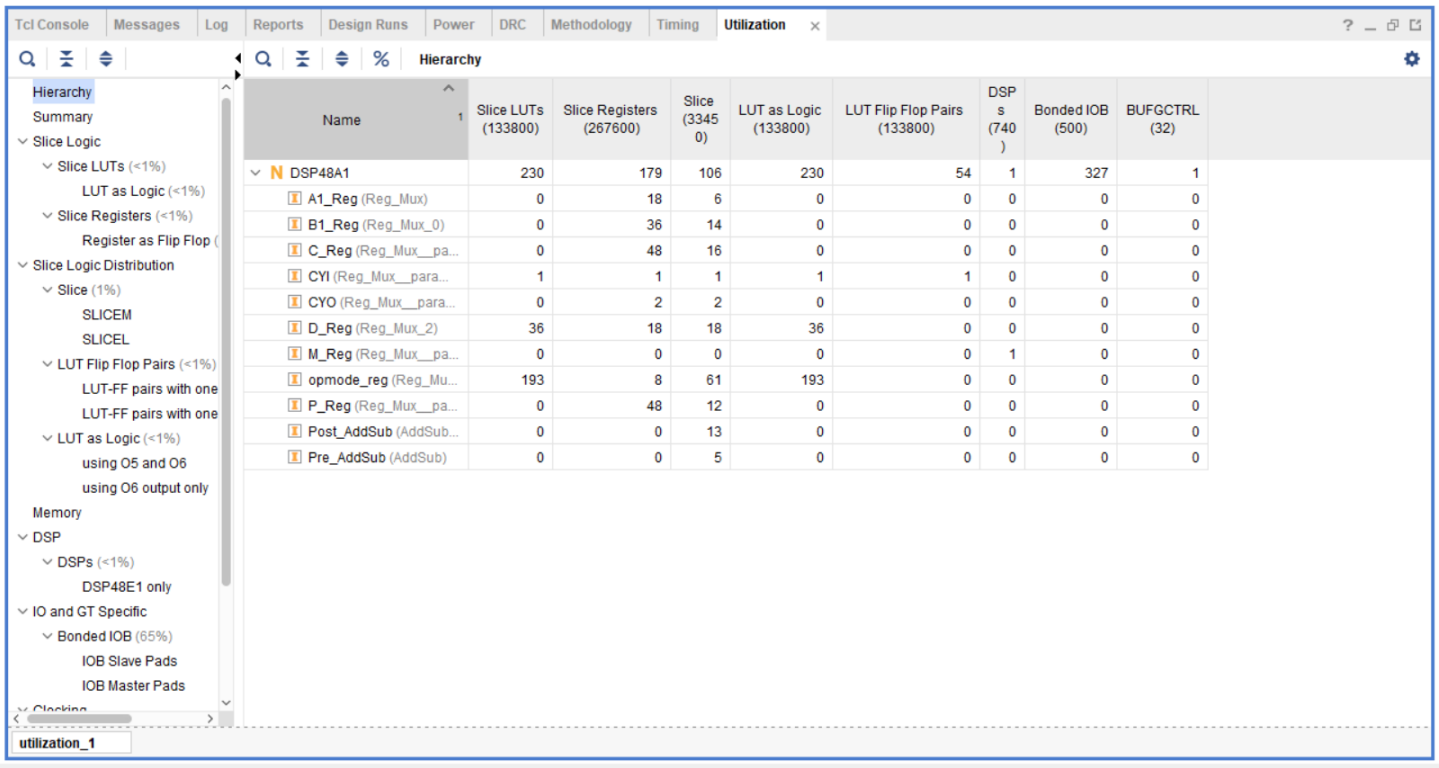


8) IMPLEMENTATION :

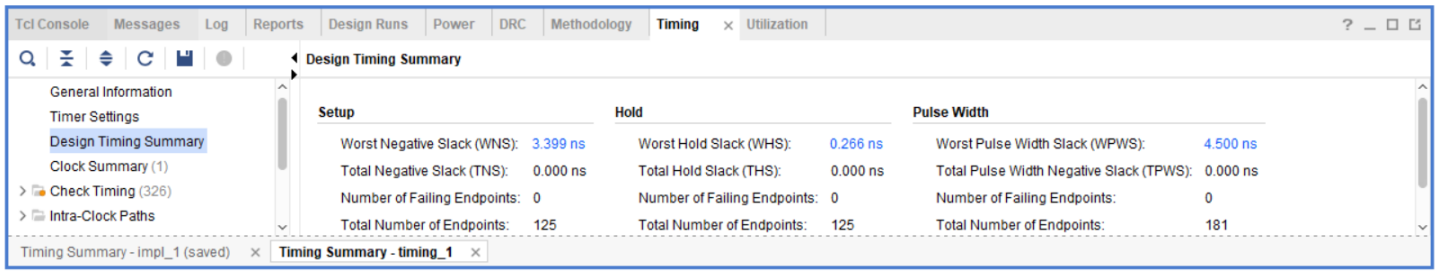
- Messages



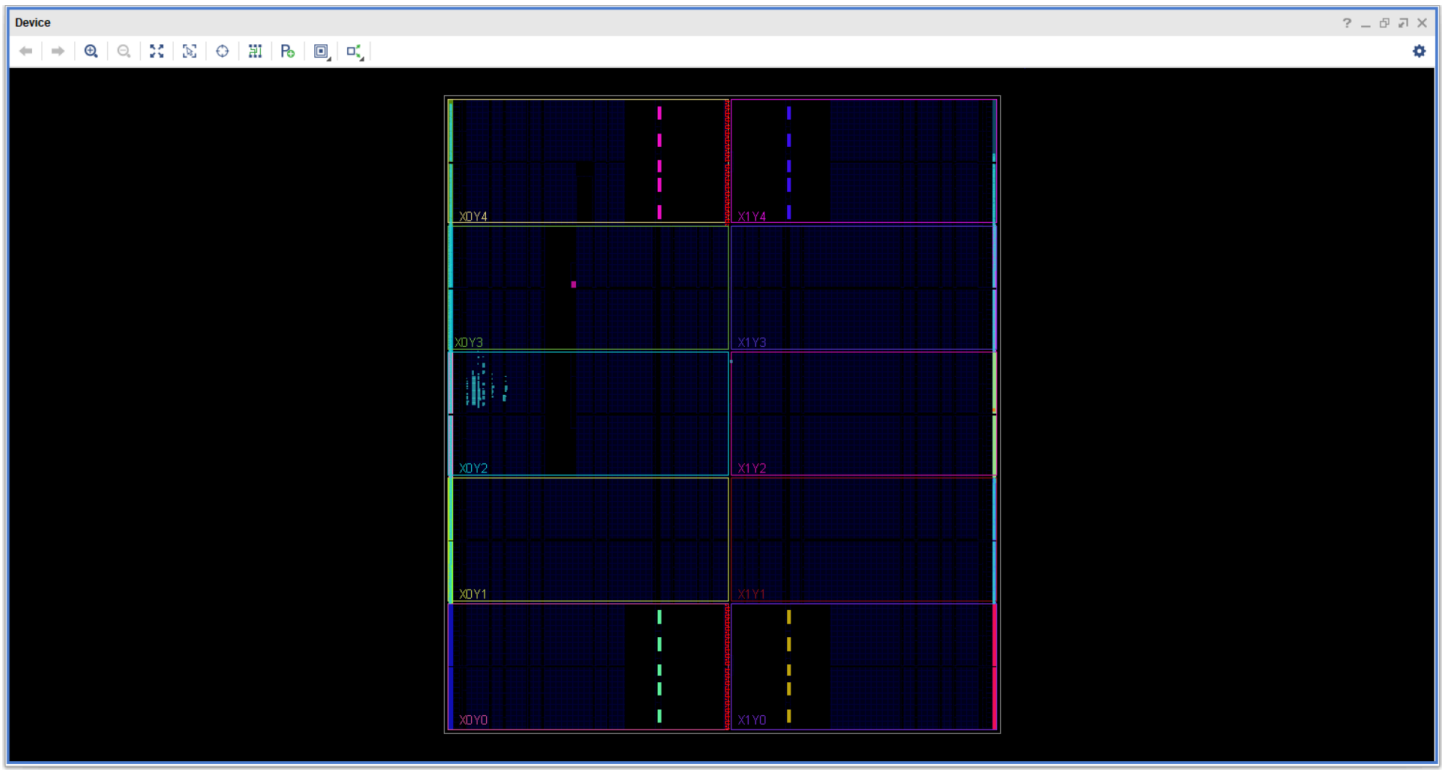
- Utilization Report



- Timing Report



- Device Snippets



9) LINTING :

- 0 Warnings, 0 Errors only 9 Info which is Waived